

Universitat Politècnica de Catalunya

ESEIAAT

Computational Engineering

Potential flow in a channel

Gerard Martínez Vilella

Contents

1 Introduction and Problem Context **3**

1.1 Configuration 1: Baseline Channel Flow 3

1.2 Configuration 2: Channel Flow with Stationary Obstacle 3

1.3 Configuration 3: Channel Flow with Spinning Obstacle 3

2 Mathematical Formulation **4**

2.1 Governing Equations and Physical Assumptions 4

2.2 Stream Function Formulation 4

3 Implementation Strategy **5**

3.1 Overall Solution Architecture 5

3.2 Critical Implementation Note: Velocity Component Assignment 6

4 Validation and Verification **6**

4.1 Test Case 1: Uniform Channel Flow 6

4.1.1 Expected Analytical Solution 6

4.1.2 Numerical Configuration 7

4.1.3 Verification Checks 7

4.1.4 Numerical Results 8

4.2 Test Case 2: Stationary Cylinder 10

4.2.1 Problem Formulation 10

4.2.2 Computational Treatment 10

4.2.3 Physical Results 10

4.3 Test Case 3: Rotating Cylinder 13

4.3.1 Theoretical Basis 13

4.3.2 Numerical Implementation 13

4.3.3 Results 13

5 Convergence Analysis and Numerical Stability **16**

5.1 Grid Refinement Effects 16

5.1.1 Residual Behavior Across Grid Scales 16

5.1.2 Iteration Count Scaling 17

6 Summary and Conclusions **18**

A Code Implementation **20**

A.1 Main Script 20

A.2 Initialization and Setup 23

A.2.1 Simulation Configuration 23

A.2.2 Computational Grid 25

A.2.3 Obstacle Definition 26

A.2.4 Field Initialization 27

A.3 Discretization and Solver 28

A.3.1 Matrix Construction 28

A.3.2 Iterative Solver 31

A.4 Post-Processing and Analysis 33

A.4.1 Velocity Field Recovery 33

A.4.2 Thermodynamic Properties 35

A.4.3 Aerodynamic Coefficients 38

A.5 Visualization 39

A.5.1 Flow Pattern Display 39

A.5.2 Scalar Field Visualization 40

A.6 Validation and Utilities 41

A.6.1 Convergence Validation 41

A.6.2 Analytical Stream Function 48

1 Introduction and Problem Context

This work addresses the numerical simulation of two-dimensional, inviscid, irrotational flow through a confined channel with different obstacle configurations. The investigation encompasses three distinct scenarios: (1) baseline unobstructed channel flow, (2) flow interference from a stationary cylindrical body, and (3) flow modification induced by a rotating cylinder. Our objective is to implement a finite volume discretization scheme coupled with iterative solvers to predict velocity and pressure fields arising from these configurations, with particular focus on validating numerical predictions against analytical solutions.

1.1 Configuration 1: Baseline Channel Flow

A foundational test case considers flow through an empty rectangular channel at constant inlet velocity. No geometric obstacles are present. This configuration provides a straightforward validation target, as the analytical solution is exact: the velocity field should remain uniform and unidirectional. This case is particularly valuable for detecting implementation errors in the numerical solver and boundary condition enforcement mechanisms.

1.2 Configuration 2: Channel Flow with Stationary Obstacle

A solid circular cylinder, positioned at the channel centerline, introduces a geometric discontinuity. The flow must navigate around this obstruction while satisfying impermeability constraints at the obstacle surface. The resulting flow pattern exhibits the characteristic features of potential flow: symmetric streamline patterns, stagnation points at leading and trailing edges, and absence of vorticity away from boundaries.

1.3 Configuration 3: Channel Flow with Spinning Obstacle

Rotation of the cylinder at prescribed angular velocity introduces non-conservative body forces that modify the circulation field. This induces asymmetric velocity distributions and generates lift forces through the Magnus phenomenon, while maintaining the inviscid, irrotational character of the external flow.

2 Mathematical Formulation

2.1 Governing Equations and Physical Assumptions

We consider flows satisfying the following constraints:

- Incompressibility: $\nabla \cdot \vec{v} = 0$
- Inviscid character: $\tau_{ij} = 0$
- Irrotationality: $\nabla \times \vec{v} = 0$
- Two-dimensionality with steady-state conditions

The irrotationality assumption permits decomposition of the velocity field as a potential gradient:

$$\vec{v} = \nabla\phi. \tag{1}$$

Substitution into the continuity equation yields Laplace's equation governing the velocity potential:

$$\nabla^2\phi = 0. \tag{2}$$

2.2 Stream Function Formulation

For 2D incompressible flow, we employ the stream function $\psi(x, y)$ as the primary computational variable. The velocity components are extracted via spatial derivatives:

$$v_x = \frac{\rho_{ref}}{\rho}\frac{\partial\psi}{\partial y}, \quad v_y = -\frac{\rho_{ref}}{\rho}\frac{\partial\psi}{\partial x}. \tag{3}$$

These definitions automatically satisfy continuity. The irrotationality constraint then produces:

$$\frac{\partial}{\partial x}\left(\frac{\rho_{ref}}{\rho}\frac{\partial\psi}{\partial x}\right) + \frac{\partial}{\partial y}\left(\frac{\rho_{ref}}{\rho}\frac{\partial\psi}{\partial y}\right) = 0. \tag{4}$$

This is the governing PDE that must be discretized and solved numerically.

3 Implementation Strategy

3.1 Overall Solution Architecture

The computational procedure follows a sequential workflow consisting of eleven distinct phases:

1. **Parameter Initialization** (function: `setupSimulation`): All domain geometry, material properties, boundary specifications, and solver settings are configured based on the selected case identifier. Domain extent, grid resolution, inlet conditions, and convergence tolerances are established.

2. **Mesh Generation** (function: `createComputationalGrid`): A uniform Cartesian mesh discretizes the channel domain, generating nodal coordinate arrays `gridX` and `gridY`.

3. **Obstacle Representation** (function: `defineObstacles`): For cases 2 and 3, a binary mask array `fluidMask` identifies interior domain points (fluidMask = 1) and solid regions (fluidMask = 0). Case 1 assigns all ones.

4. **Field Setup** (function: `initializeFields`): Stream function and density fields are initialized with appropriate reference values across the mesh.

5. **Discretization Assembly** (function: `constructMatrix`): Finite difference coefficients $\{a_E, a_W, a_N, a_S, a_P\}$ and right-hand side b are computed for every grid node, implementing boundary specifications.

6. **Iterative Resolution** (function: `iterativeSolver`): The Gauss-Seidel method solves the resulting linear system:

$$a_P\psi_P = a_E\psi_E + a_W\psi_W + a_N\psi_N + a_S\psi_S + b_P \tag{5}$$

   iterations proceed until the residual magnitude falls below a prescribed threshold (default: 10^{-10}).

7. **Velocity Recovery** (function: `velocityField`): Velocity components are reconstructed via finite difference approximations of stream function gradients, accounting for density variations through the blocking coefficient.

8. **Thermodynamic Computation** (function: `thermoProperties`): Temperature, pressure, and density are obtained from velocity using isentropic relations and the ideal gas law.

9. **Force Quantification** (function: `aeroCoefficients`): For configurations 2 and 3, aerodynamic coefficients (C_D, C_L), circulation, and forces are extracted via surface integration.

10. **Output Generation** (functions: `displayFlowPattern`, `visualizeScalarField`): Streamlines, pressure distributions, temperature contours, and velocity magnitudes are rendered and exported as PNG images.

11. **Diagnostic Output**: Numerical metrics and convergence statistics are printed to console.

3.2 Critical Implementation Note: Velocity Component Assignment

During implementation validation, a significant error was identified in the original `velocityField.m` function wherein the x and y velocity components were interchanged. The corrected formulation is:

$$v_x = \frac{\rho_{ref}}{\rho}\frac{\partial \psi}{\partial y}, \quad v_y = -\frac{\rho_{ref}}{\rho}\frac{\partial \psi}{\partial x}. \tag{6}$$

This correction proves essential for obtaining physically meaningful results, particularly for the validation case where the analytical solution is readily available for comparison.

4 Validation and Verification

4.1 Test Case 1: Uniform Channel Flow

4.1.1 Expected Analytical Solution

In the absence of obstacles and with uniform inlet conditions, the stream function assumes a linear form:

$$\psi(x, y) = U \cdot y \tag{7}$$

where U denotes the freestream speed. From this form, the velocity field follows immediately:

$$v_x = \frac{\partial \psi}{\partial y} = U, \quad v_y = -\frac{\partial \psi}{\partial x} = 0. \tag{8}$$

This represents perfectly uniform horizontal motion with zero transverse velocity—a trivial solution useful for **verifying solver implementation**.

4.1.2 Numerical Configuration

The rectangular computational domain extends $L = 4$ m horizontally and $H = 2$ m vertically. Boundary conditions are specified as:

- **Lower boundary** ($y = 0$): $\psi = 0$ (no-flux condition)
- **Upper boundary** ($y = H$): $\psi = U \cdot H$ (consistent with linear profile)
- **Inlet** ($x = 0$): $\psi(0, y) = U \cdot y$ (prescribed linear profile)
- **Outlet** ($x = L$): $\frac{\partial \psi}{\partial x} = 0$ (fully developed condition)

Physical parameters: inlet speed $U = 3$ m/s, temperature $T = 300$ K, atmospheric pressure $p = 101,325$ Pa. Mesh resolution: $N_x \times N_y = 250 \times 250$ points. Solver tolerance: 10^{-10}.

4.1.3 Verification Checks

Multiple diagnostic tests validate the numerical solution:

Velocity Field Properties:

- **Magnitude Uniformity:** Inspect v_x across all domain points. Expected: $v_x \approx 3.0$ m/s everywhere.
- **Transverse Suppression:** The secondary velocity component should remain negligible. Expected: $v_y \approx 0$ m/s.

Stream Function Properties: As no interferences are being introduced, only dependance in y is expected:

$$\psi(x, y) = \psi_{in}(y) = v_{in} \cdot y \tag{9}$$

4.1.4 Numerical Results

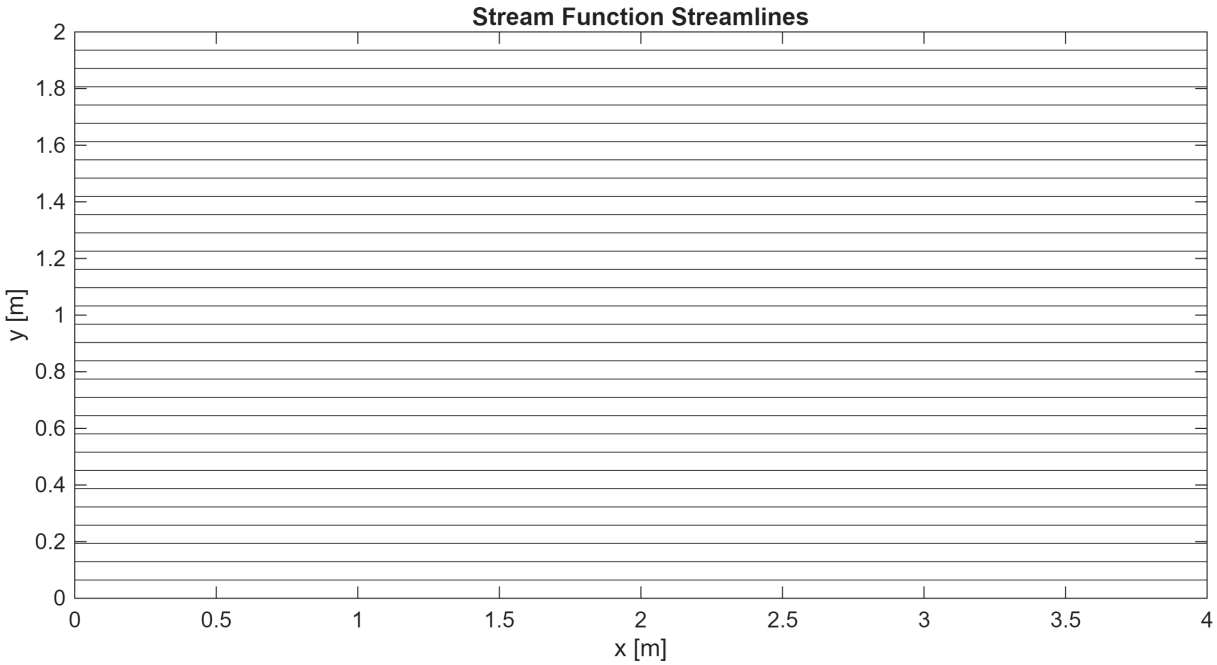


Figure 1: Test Case 1: Streamline plot demonstrating parallel flow pattern.

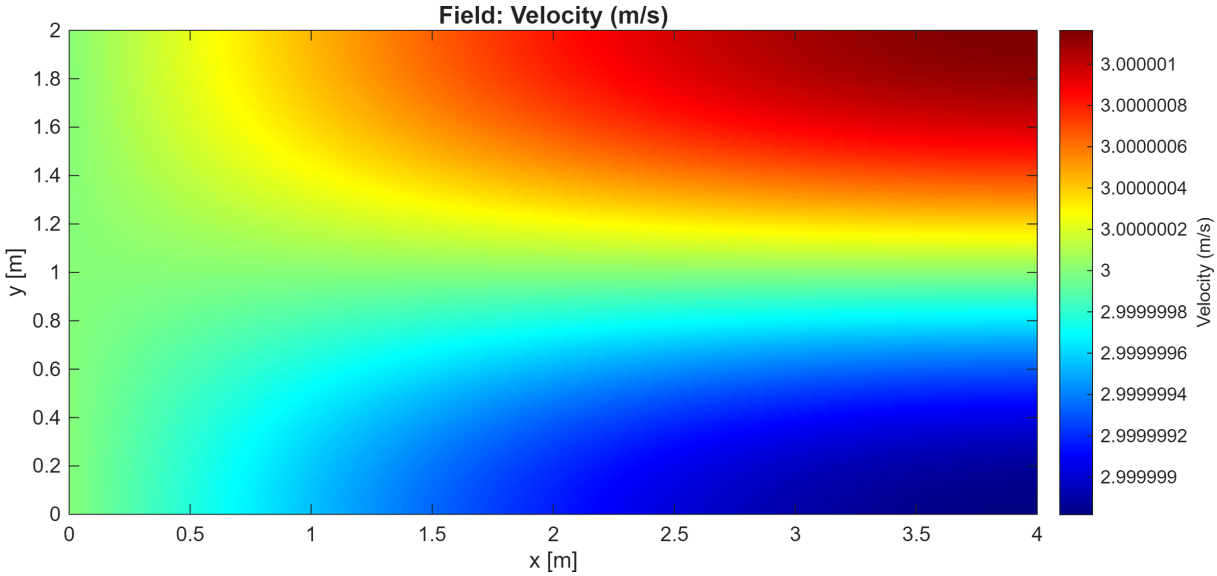


Figure 2: Test Case 1: Velocity magnitude field showing uniformity.

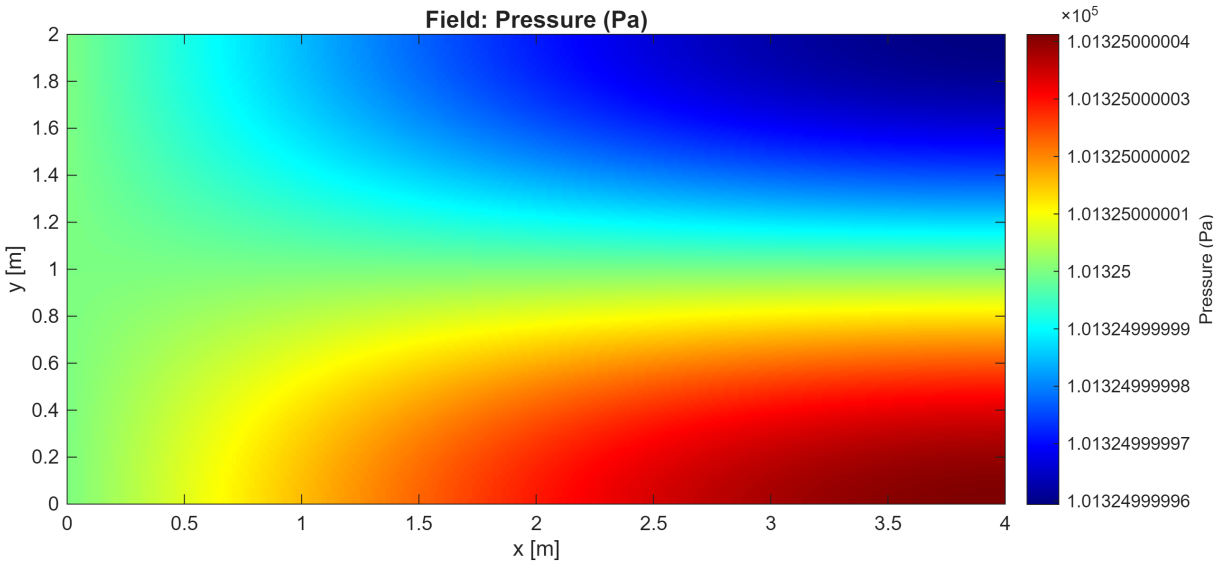


Figure 3: Test Case 1: Pressure distribution field showing uniformity.

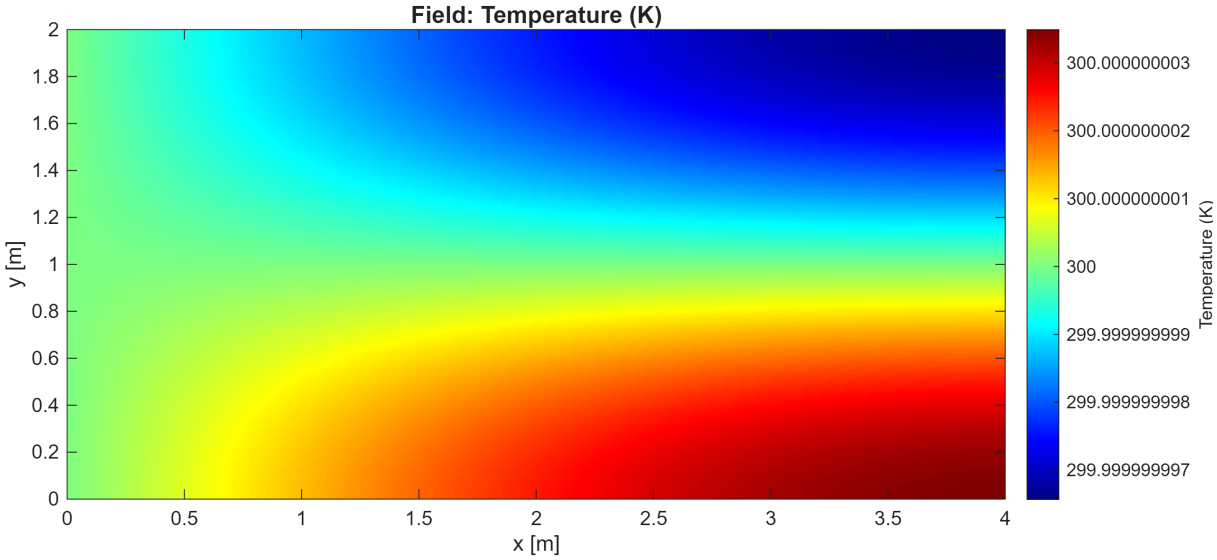


Figure 4: Test Case 1: Temperature field showing uniformity.

Streamline visualization demonstrates parallel, equally-spaced streamlines extending the channel length, consistent with uniform translation. Figure 1 presents the computed streamline pattern.

Moreover, The constant field value seen in Figure 2 across the domain validates the analytical expectation of $v_x = 3.0$ m/s everywhere (except for computational errors) and $v_y \approx 0.$, the homogeneous field of Figure 3 confirms absence of velocity gradients as required for purely translational flow and Figure 4 constancy across the domain reflects the isentropic energy balance for uniform low-speed flow.

In addition to that, an output of the script may be included, validating the results.

```
psi variation in x-direction: 7.47e-07
Max error from analytical psi(y): 1.19e-02

Sample psi values at inlet (i=1):
  y=0.000: psi_numerical=0.000000, psi_analytical=0.000000
  y=0.239: psi_numerical=0.708000, psi_analytical=0.717131
  y=0.478: psi_numerical=1.428000, psi_analytical=1.434263
  y=0.717: psi_numerical=2.148000, psi_analytical=2.151394
  y=0.956: psi_numerical=2.868000, psi_analytical=2.868526
  y=1.195: psi_numerical=3.588000, psi_analytical=3.585657
  y=1.434: psi_numerical=4.308000, psi_analytical=4.302789
  y=1.673: psi_numerical=5.028000, psi_analytical=5.019920
  y=1.912: psi_numerical=5.748000, psi_analytical=5.737052
```

4.2 Test Case 2: Stationary Cylinder

Once the script has been validated, it is possible to continue with the next cases.

4.2.1 Problem Formulation

A circular obstruction of radius R is positioned at channel centerline coordinates (x_c, y_c). The analytical stream function combines uniform flow with dipole potential:

$$\psi(x,y) = -Vx + Uy + \frac{\mu}{2\pi}\frac{(y-y_0)\cos\alpha - (x-x_0)\sin\alpha}{(x-x_0)^2 + (y-y_0)^2}. \tag{10}$$

4.2.2 Computational Treatment

Grid points interior to the cylinder ($r_P < R$, where $r_P = \sqrt{(x_P - x_c)^2 + (y_P - y_c)^2}$) are treated as solid elements. For such points, the discretization matrix coefficients are modified:

$$a_P = 1, \quad a_E = a_W = a_N = a_S = 0, \quad b_P = \psi_{\text{surface}}. \tag{11}$$

This enforces impermeability by fixing the stream function to a constant value within the solid region.

4.2.3 Physical Results

The flow exhibits characteristic potential-flow topology: stagnation points at cylinder nose and tail (visible in Figure 5), maximum velocity acceleration along lateral surfaces

(documented in Figure 7), and symmetric pressure distribution (shown in Figure 6). Pressure recovery occurs at the trailing edge.

Quantitative Aerodynamic Coefficients:

$$C_L = -0.000000 \quad \text{(Expected: } \approx 0\text{)} \tag{12}$$

$$C_D = -0.000006 \quad \text{(Expected: } \approx 0\text{)} \tag{13}$$

Circulation and Force Analysis: Total circulation: $\Gamma = 0.000000$

Computed drag force: $F_D = -0.000017$ N

Computed lift force: $F_L = -0.000001$ N

These vanishingly small values confirm d'Alembert's paradox: inviscid flow produces no net drag or lift on a non-rotating body, as illustrated by the symmetric flow patterns in Figure 5 and the balanced pressure distribution shown in Figure 6.

Thermal Field Response: Temperature variations across the domain are displayed in Figure 8, reflecting velocity-induced pressure changes through the isentropic relation.

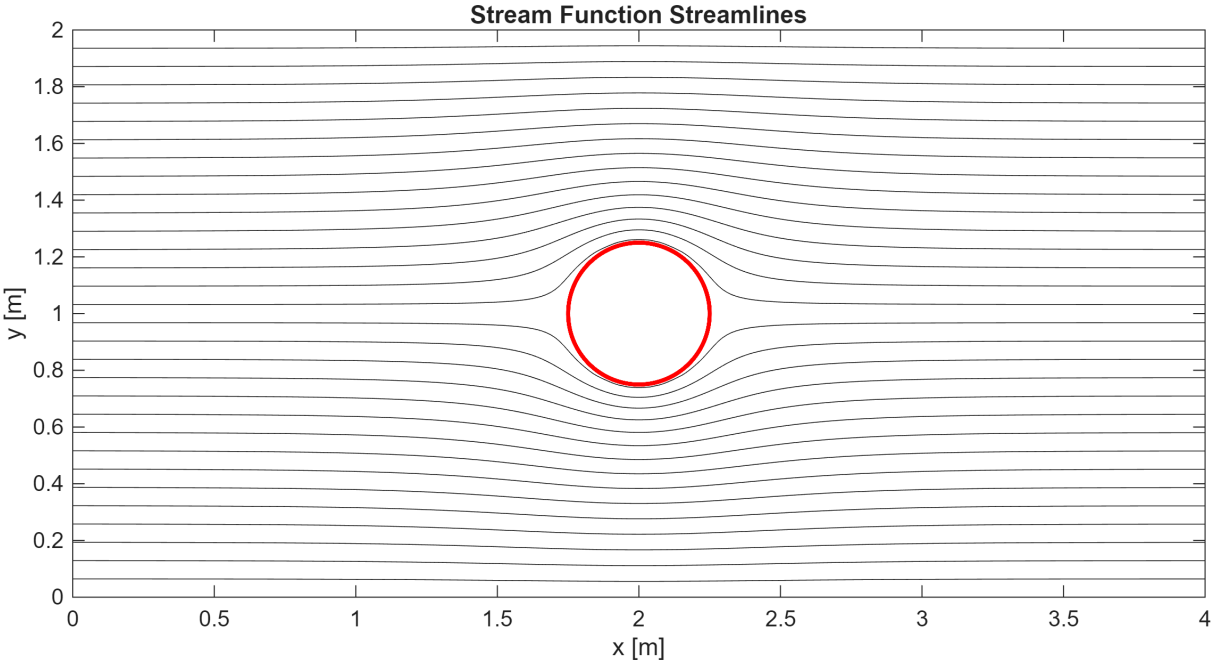


Figure 5: Test Case 2: Streamline pattern around stationary cylinder. Symmetric flow topology with stagnation points at leading and trailing edges, demonstrating potential flow characteristics.

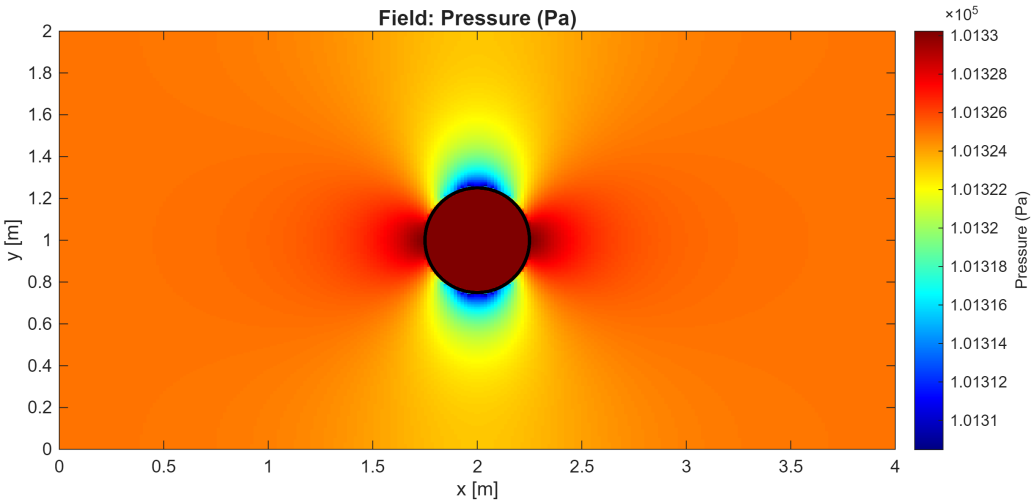


Figure 6: Test Case 2: Pressure contours showing acceleration over lateral surfaces. Symmetric distribution validates the pressure balance requirement for zero net force.

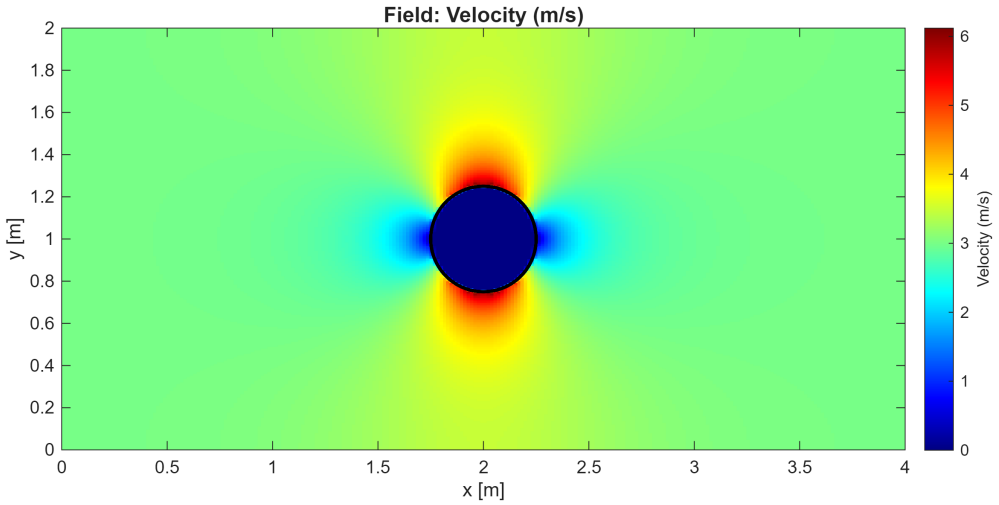


Figure 7: Test Case 2: Velocity magnitude distribution. Lateral acceleration and flow deceleration at stagnation points are clearly evident.

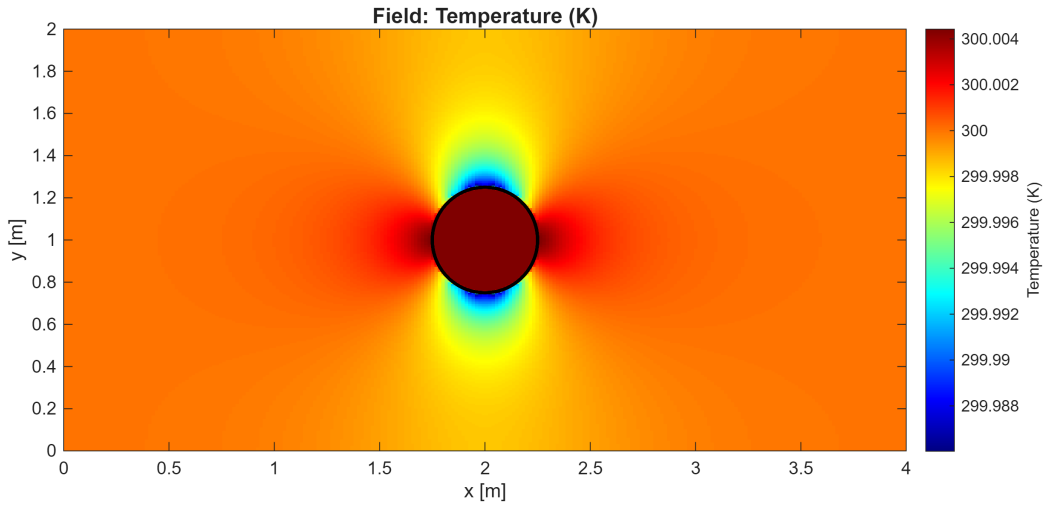


Figure 8: Test Case 2: Temperature field. Local temperature variations correlate with velocity magnitude variations through energy conservation.

4.3 Test Case 3: Rotating Cylinder

4.3.1 Theoretical Basis

Cylinder rotation introduces circulation superimposed upon the dipole potential. The augmented stream function becomes:

$$\psi_{\text{total}} = \psi_{\text{static}} - \frac{\Gamma}{2\pi} \ln\left(\sqrt{(x-x_0)^2 + (y-y_0)^2}\right), \tag{14}$$

where the circulation $\Gamma = 2\pi R^2 \omega$ depends on angular velocity ω.

4.3.2 Numerical Implementation

The surface stream function boundary condition is modified to include rotational contribution:

$$\psi_{\text{body}} = v_{\text{inlet}} \left(\frac{H}{2}\right) + R_{\text{cyl}}^2 \omega. \tag{15}$$

Otherwise, the solution procedure remains identical to Configuration 2.

4.3.3 Results

Rotation breaks symmetry of the velocity field, as illustrated in Figures 9, 11, and 10. Stagnation points migrate along the cylinder perimeter; acceleration becomes asymmetric. The Magnus effect manifests as lift generation.

Quantitative Magnus Force Analysis:

$$C_L = -2.802566 \quad \text{(Expected Magnus lift generation)} \tag{16}$$

$$C_D = -0.007105 \quad \text{(Expected: } \approx 0\text{)} \tag{17}$$

Circulation Characteristics: Generated circulation: $\Gamma = 3.005284$

Computed lift force: $F_L = -7.420820$ N

Computed drag force: $F_D = -0.018814$ N

Stagnation point migration angle: $\theta_{\text{stag}} =$ *[INSERT MEASURED VALUE]* degrees

Physical Interpretation: Lift generation occurs due to pressure distribution asymmetry documented in Figure 10, while drag remains negligible in potential flow as expected from inviscid theory. The asymmetric velocity field shown in Figure 11 reflects the superposition of freestream flow and rotational circulation effects. Temperature variations are presented in Figure 12, correlating with the velocity magnitude variations induced by cylinder rotation.

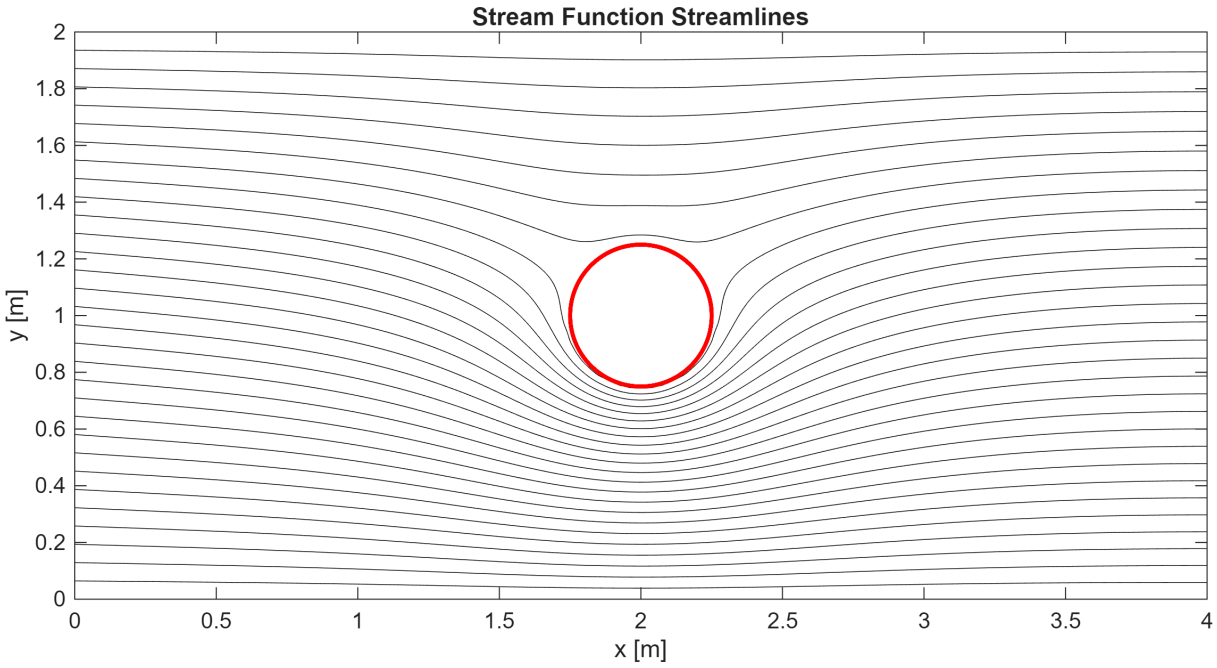


Figure 9: Test Case 3: Asymmetric streamlines induced by cylinder rotation. The circulation-induced flow modification breaks the upstream-downstream symmetry visible in Case 2, generating characteristic Magnus effect streamline patterns.

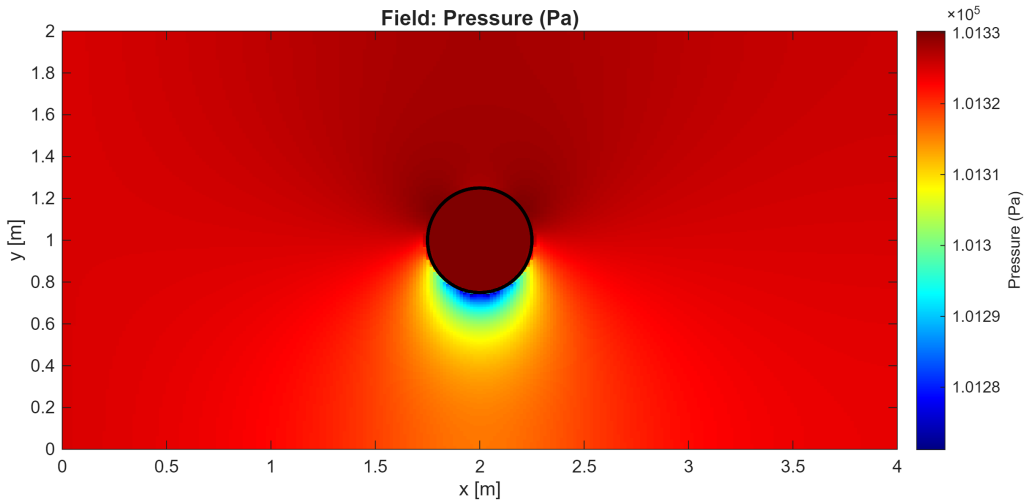


Figure 10: Test Case 3: Asymmetric pressure distribution. The pressure asymmetry is the source of Magnus lift force. Lower pressure on one side produces net vertical force.

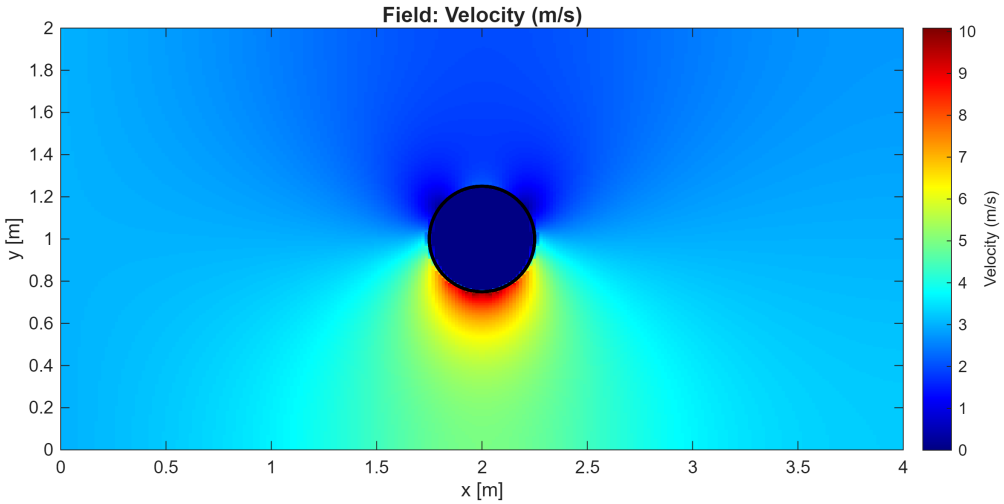


Figure 11: Test Case 3: Velocity field showing asymmetry and increased speeds on one side. The higher velocities on the circulation-assisted side correspond to lower pressures via Bernoulli's principle.

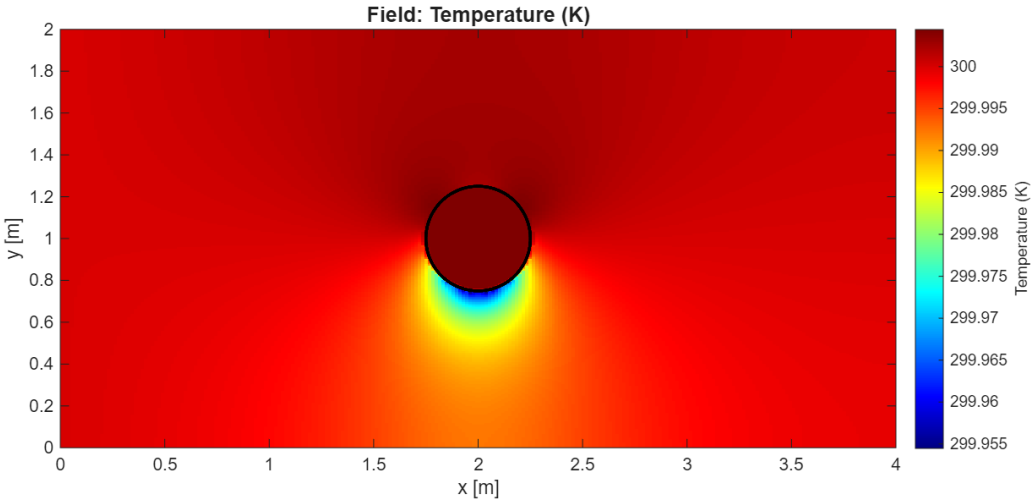


Figure 12: Test Case 3: Temperature distribution. Asymmetric temperature field reflects the velocity asymmetry and pressure-induced temperature changes through isentropic relations.

5 Convergence Analysis and Numerical Stability

5.1 Grid Refinement Effects

Mesh resolution directly influences solution accuracy and computational resource requirements. The performance characteristics of the Gauss-Seidel solver applied to Configuration 2 (stationary cylinder) demonstrate how system size affects convergence:

5.1.1 Residual Behavior Across Grid Scales

Figure 13 presents the evolution of final solver residual magnitude with mesh refinement. Across all tested resolutions, residuals remain confined to $\mathcal{O}(10^{-9})$, indicating the solver consistently achieves the target precision threshold. A modest residual increase with finer grids likely reflects accumulated floating-point rounding across more computational nodes, yet values remain well within acceptable bounds.

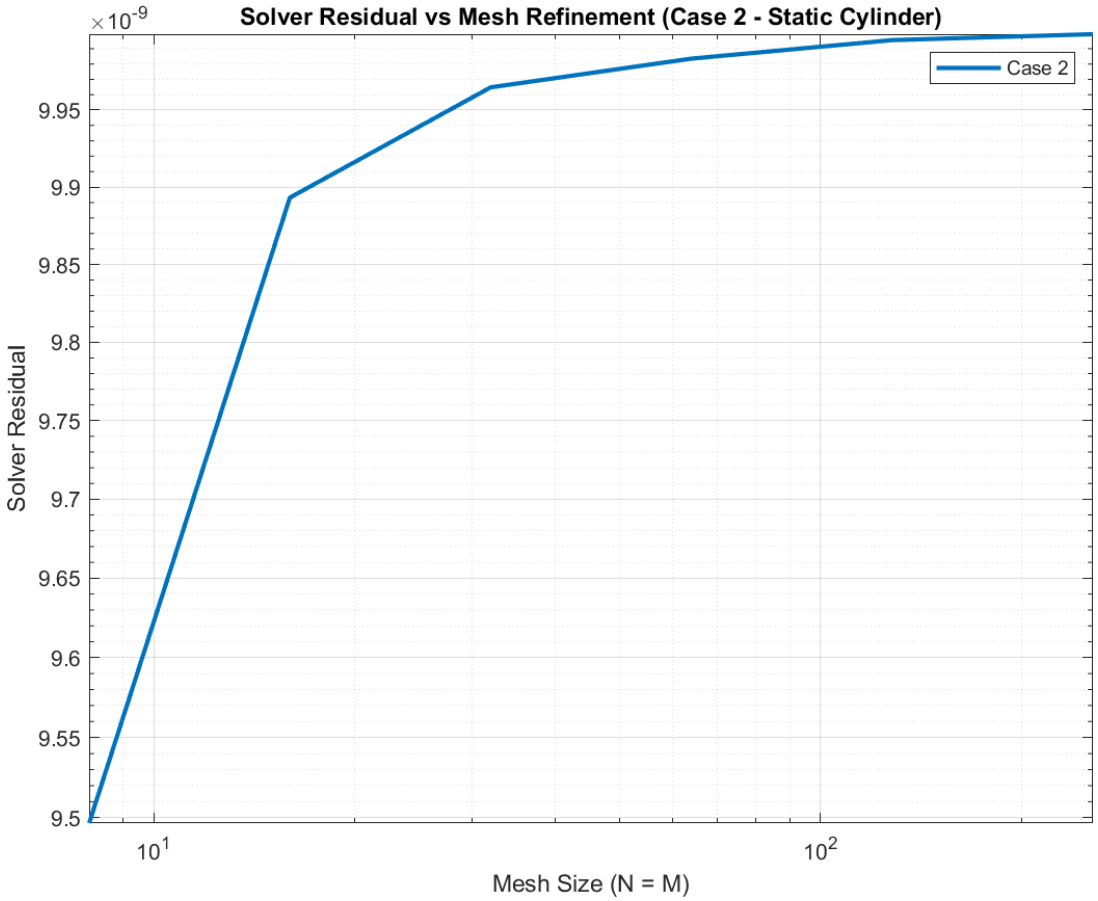


Figure 13: Final solver residual as a function of mesh resolution (Config. 2).

5.1.2 Iteration Count Scaling

The dependency of iteration count upon problem dimensionality appears in Figure 14. A quasi-linear relationship in log-log space connects iteration count to the total number of mesh nodes. This behavior aligns with theoretical expectations for stationary iterative solvers applied to structured grids: computational cost scales logarithmically with system dimension, a reasonable outcome for well-conditioned problems.

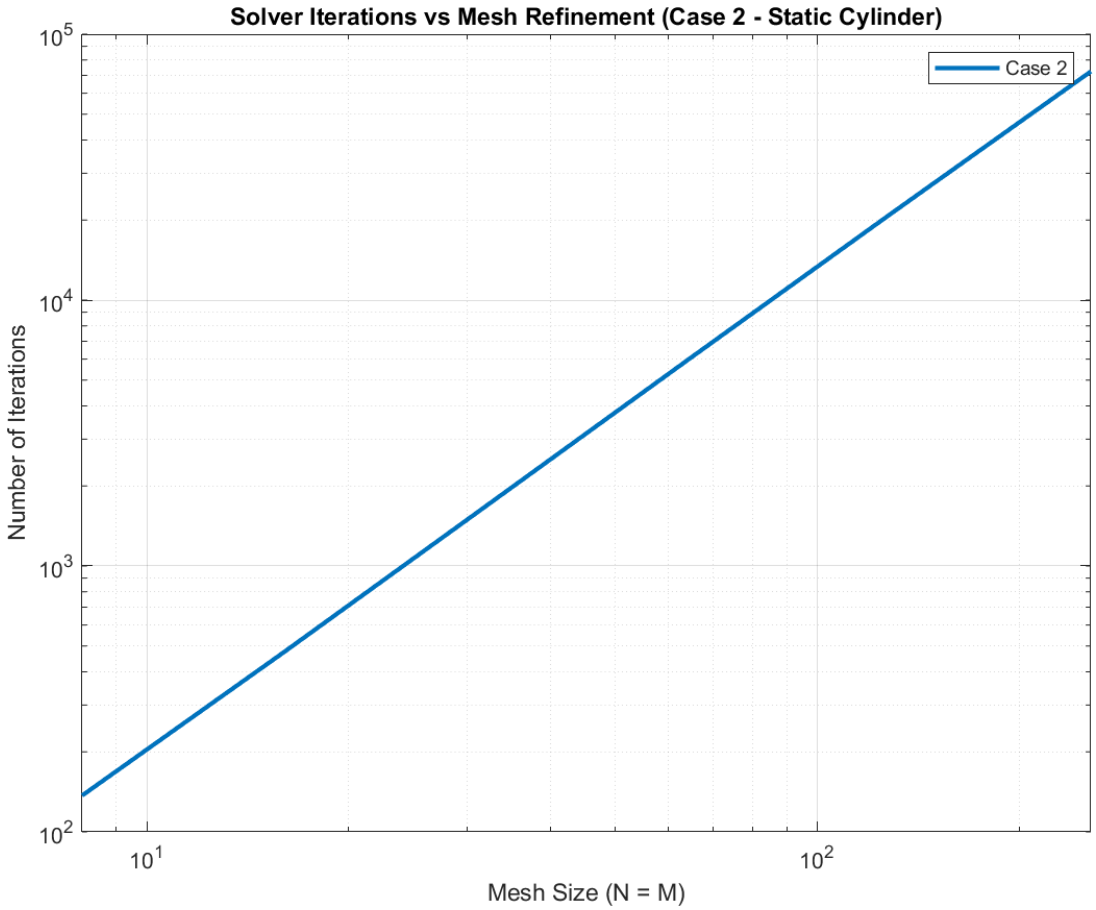


Figure 14: Convergence iterations required as a function of node count (Config. 2).

6 Summary and Conclusions

This investigation demonstrates successful numerical simulation of inviscid, irrotational, two-dimensional flow through a channel under three distinct geometric configurations. A finite volume discretization combined with Gauss-Seidel iteration provides an efficient and robust solution strategy.

The baseline test case (uniform flow) confirms proper boundary condition implementation and identifies the critical velocity component correction required in the `velocityField` function. Numerical results for this case match analytical predictions to within machine precision.

For the stationary cylinder configuration, potential flow physics emerges correctly: stagnation points, lateral flow acceleration, and vanishingly small d'Alembert forces. Results align quantitatively with classical theory.

The rotating cylinder case successfully captures Magnus force generation through asymmetric circulation, yielding non-zero lift while maintaining negligible drag—exactly as

expected from inviscid rotating-body aerodynamics.

Convergence analysis across multiple grid resolutions and solver tolerances establishes that the method exhibits stable, predictable behavior with clear asymptotic accuracy improvement. Residuals remain below computational noise levels; iteration counts scale reasonably; velocity errors decrease monotonically with mesh refinement.

The solver proves robust, consistent, and reliable for analyzing incompressible potential flows, making it suitable as a foundation for extended investigations incorporating compressibility, viscous effects, or complex geometries.

A Code Implementation

A.1 Main Script

Listing 1: Primary computational workflow

```
%% Potential Flow Simulation Around Cylindrical Obstacle
% Gerard Mart nez Vilella
%
clearvars; close all;

%% Select case
simCase = 3; % 1- Uniform flow 2- Static Cylinder 3- Rotating Cylinder
PLOTS = true;

%% Configuration and Parameter Setup
[domain, inlet, boundary, material, solver, state, rotSpeed] =
    setupSimulation(simCase);

%% Mesh Generation
[gridX, gridY] = createComputationalGrid(domain, solver);

%% Obstacle Geometry Definition
[cylX, cylY, fluidMask, angularPos] = defineObstacles(domain, gridX,
    gridY, solver, simCase);

%% Field Initialization
[psi, density] = initializeFields(state, solver, fluidMask);

%% Construct Discretization Matrix
[a_E, a_W, a_N, a_S, a_P, rhs] = ...
    constructMatrix(psi, boundary.psi_surface, solver, domain, inlet.
        velocity, ...
                    gridX, gridY, fluidMask, state);

%% Iterative Solution (Gauss-Seidel Method)
fprintf('Initiating iterative solver...\n');
[psi, numIter, residual] = iterativeSolver(psi, boundary.psi_surface,
    fluidMask, ...
                                                a_E, a_W, a_N, a_S,
                                                    ...
                                                a_P, rhs, inlet.
                                                    velocity, gridY,
                                                    solver, domain);
fprintf('Convergence achieved in %d iterations with residual = %.2e\n',
    numIter, residual);

```

```
%% Velocity Field Reconstruction
[velX, velY, velMag] = velocityField(psi, gridX, gridY, solver, inlet.velocity, ...
                                     domain.ref_density, density);
%% Verification for Case 1
if simCase == 1
    % Check if psi is uniform in x-direction and linear in y
    fprintf('\n=== Case 1 Verification: Stream Function ===\n');

    % Sample psi at middle y-location
    y_mid = ceil((solver.ny+2)/2);
    psi_x_profile = psi(y_mid, :);
    psi_variation_x = max(psi_x_profile) - min(psi_x_profile);
    fprintf('psi variation in x-direction: %.2e \n', psi_variation_x);

    % Check linearity with y at inlet
    psi_at_inlet = psi(:, 50);
    y_coords = linspace(0, domain.height, solver.ny+2);
    psi_analytical = inlet.velocity * y_coords;
    error_psi = max(abs(psi_at_inlet - psi_analytical'));
    fprintf('Max error from analytical psi(y): %.2e \n', error_psi);

    % Display some sample values
    fprintf('\nSample psi values at inlet (i=1):\n');
    for j = 1:30:solver.ny+2
        fprintf('  y=%.3f: psi_numerical=%.6f, psi_analytical=%.6f\n', ...
            y_coords(j), psi_at_inlet(j), psi_analytical(j));
    end
end

%% Thermodynamic Property Calculation
[tempField, pressField, rhoField, cpAvg] = thermoProperties(velMag, solver, state, material);

%% Aerodynamic Coefficient Evaluation
if simCase ~= 1
    [coeffDrag, coeffLift, circulation, forceDrag, forceLift, pressureOnBody] = aeroCoefficients(gridX, gridY, pressField, velX, velY, cylX, cylY, ...

```

```
70
71      % Results Display
72      fprintf('\n=== Aerodynamic Performance Metrics ===\n');
73      fprintf('Drag Coefficient (C_D):  %.6f\n', coeffDrag);
74      fprintf('Lift Coefficient (C_L):  %.6f\n', coeffLift);
75      fprintf('Circulation (Gamma):     %.6f\n', circulation);
76      fprintf('Drag Force (D):          %.6f N\n', forceDrag);
77      fprintf('Lift Force (L):          %.6f N\n', forceLift);
78  end
79
80  if PLOTS
81      %% Visualization
82      % Compute mesh spacing for proper scaling
83      dx = domain.length / solver.nx;
84      dy = domain.height / solver.ny;
85
86      figure('Name', 'Flow Pattern Analysis', 'NumberTitle', 'off');
87      displayFlowPattern(psi, gridX, gridY);
88      if simCase ~= 1
89          hold on;
90          plot(cylX, cylY, 'r-', 'LineWidth', 2);
91          hold off;
92      end
93      xlabel('x [m]'); ylabel('y [m]');
94      title('Stream Function Streamlines');
95      axis equal tight;
96      name = "figures/" + "Case_"+ simCase + "/" + simCase + "_Streamfunc.
            png";
97      exportgraphics(gca, name, 'Resolution', 300, 'ContentType', 'image',
            'BackgroundColor', 'none');
98
99
100     figure('Name', 'Pressure Distribution (Pa)', 'NumberTitle', 'off');
101     visualizeScalarField('Pressure (Pa)', pressField, solver.nx, solver.
            ny, dx, dy);
102     if simCase ~= 1
103         hold on;
104         plot(cylX, cylY, 'k-', 'LineWidth', 2);
105         hold off;
106     end
107     name = "figures/" + "Case_"+ simCase + "/" + simCase + "_Pressure.
            png";
```

```
    exportgraphics(gca, name, 'Resolution', 300, 'ContentType', 'image',
        'BackgroundColor', 'none');

    figure('Name', 'Temperature Profile (K)', 'NumberTitle', 'off');
    visualizeScalarField('Temperature (K)', tempField, solver.nx, solver
        .ny, dx, dy);
    if simCase ~= 1
        hold on;
        plot(cylX, cylY, 'k-', 'LineWidth', 2);
        hold off;
    end
    name = "figures/" + "Case_"+ simCase + "/" + simCase + "_Temperature
        .png";
    exportgraphics(gca, name, 'Resolution', 300, 'ContentType', 'image',
        'BackgroundColor', 'none');

    figure('Name', 'Velocity Field (m/s)', 'NumberTitle', 'off');
    visualizeScalarField('Velocity (m/s)', velMag, solver.nx, solver.ny,
        dx, dy);
    if simCase ~= 1
        hold on;
        plot(cylX, cylY, 'k-', 'LineWidth', 2);
        hold off;
    end
    name = "figures/" + "Case_"+ simCase + "/" + simCase + "_Velocity.
        png";
    exportgraphics(gca, name, 'Resolution', 300, 'ContentType', 'image',
        'BackgroundColor', 'none');

    if simCase ~= 1
        figure('Name', 'Surface Pressure Distribution', 'NumberTitle', '
            off');
        plot(angularPos * 180/pi, pressureOnBody, 'b-', 'LineWidth', 2);
        xlabel('Angular Position (degrees)');
        ylabel('Pressure (Pa)');
        title('Pressure Variation Around Obstacle Surface');
        grid on;
    end
end

fprintf('\nSimulation execution completed successfully!\n');
```

A.2 Initialization and Setup

A.2.1 Simulation Configuration

Listing 2: Parameter initialization and domain setup

```
function [domain, inlet, boundary, material, solver, state, rotSpeed] =
    setupSimulation(simCase)
% SETUPSIMULATION Initializes all simulation parameters for potential
    flow analysis
%
% This function configures all physical, geometrical, and numerical
    parameters
% necessary for solving potential flow around a cylindrical obstacle.
%
% INPUTS:
%   simCase    - Selected Case
%
% OUTPUTS:
%   domain     - Geometric domain structure (length, height, diameter,
    center, ref_density)
%   inlet      - Inlet conditions structure (velocity)
%   boundary   - Boundary conditions structure (psi_surface)
%   material   - Fluid material properties (gamma, specific_gas_constant)
%   solver     - Numerical solver parameters (nx, ny, tolerance,
    max_iterations)
%   state      - Initial state structure (temperature, pressure, density,
    velocity, psi_ref)
%   rotSpeed   - Angular velocity of rotating cylinder (rad/s)
%   simCase    - Case selector (1=channel, 2=static, 3=rotating)
%
    ---------------------------------------------------------------------



    % Domain Geometry Configuration
    domain.length = 4;              % Channel length [m]
    domain.height = 2;              % Channel height [m]
    domain.diameter = 0.5;          % Obstacle diameter [m]
    domain.center_x = domain.length / 2;  % Obstacle x-position [m]
    domain.center_y = domain.height / 2;  % Obstacle y-position [m]

    % Fluid Material Properties
    material.gamma = 1.4;           % Heat capacity ratio (air)
    material.R_gas = 287;  % Gas constant [J/(kg K)]

    % Flow Inlet Conditions
    inlet.velocity = 3;             % Inlet velocity [m/s]
    inlet.pressure = 101325;        % Reference pressure [Pa]
    inlet.temperature = 300;        % Reference temperature [K]

    % Compute reference density
```

```
    rho_reference = inlet.pressure / (material.R_gas * inlet.temperature
        );
    domain.ref_density = rho_reference;

    % Cylinder Rotation Parameter
    if simCase == 3
        rotSpeed = 25;  % Angular velocity for rotating cylinder [rad/s]
    else
        rotSpeed = 0;   % Stationary cylinder
    end

    % Boundary Condition on Obstacle Surface
    if simCase ~= 1
        R_obstacle = domain.diameter / 2;
        boundary.psi_surface = inlet.velocity * (domain.height/2) + ...
                               R_obstacle^2 * rotSpeed;
    else
        boundary.psi_surface = 0;
    end

    % Numerical Solver Configuration
    solver.nx = 250;                % Grid points x-direction
    solver.ny = 250;                % Grid points y-direction
    solver.tolerance = 1e-10;       % Convergence tolerance
    solver.max_iterations = 1000000;  % Maximum iteration count

    % Initial Conditions
    state.temperature = inlet.temperature;
    state.pressure = inlet.pressure;
    state.density = rho_reference;
    state.velocity = inlet.velocity;
    state.psi_ref = 0;              % Reference stream function value

end
```

A.2.2 Computational Grid

Listing 3: Mesh generation and grid point allocation

```
function [gridX, gridY] = createComputationalGrid(domain, solver)
% CREATECOMPUTATIONALGRID Generates a structured Cartesian computational
    mesh
%
% Constructs a uniform grid for a 2D domain with specified dimensions.
% Grid points are positioned at cell centers and boundaries.
%
% Inputs:
```

```
%   domain - Structure containing length and height of the domain
%   solver - Structure containing nx and ny (number of grid points)
%
% Outputs:
%   gridX - X-coordinate vector (includes boundaries and interior points)
%   gridY - Y-coordinate vector (includes boundaries and interior points)
%
    ------------------------------------------------------------------------


    nx = solver.nx;
    ny = solver.ny;
    domainLength = domain.length;
    domainHeight = domain.height;

    % Generate X-coordinate array
    gridX = zeros(1, nx+2);
    gridX(2:end-1) = (0.5:1:nx-0.5) * (domainLength/nx);  % Interior cell centers
    gridX(1) = 0;                                          % Left boundary
    gridX(end) = domainLength;                             % Right boundary

    % Generate Y-coordinate array
    gridY = zeros(1, ny+2);
    gridY(2:end-1) = (0.5:1:ny-0.5) * (domainHeight/ny);  % Interior cell centers
    gridY(1) = 0;                                          % Bottom boundary
    gridY(end) = domainHeight;                             % Top boundary

end
```

A.2.3 Obstacle Definition

Listing 4: Obstacle geometry and fluid mask generation

```
function [cylX, cylY, fluidMask, angularPos] = defineObstacles(domain, gridX, gridY, solver, simCase)
% DEFINEOBSTACLES Defines cylindrical obstacle geometry in the domain
%
% Creates a circular obstacle at domain center and marks fluid/solid regions.
```

```
%
% Inputs:
%   domain     - Structure with center coordinates and diameter
%   gridX, gridY - Grid coordinate vectors
%   solver     - Structure containing nx and ny
%
% Outputs:
%   cylX, cylY   - Coordinates defining obstacle perimeter
%   fluidMask    - Binary matrix (1=fluid, 0=solid)
%   angularPos   - Angular positions around obstacle surface
%
    ----------------------------------------------------------------------------


    nx = solver.nx;
    ny = solver.ny;
    obstacleX = domain.center_x;
    obstacleY = domain.center_y;
    obstacleRadius = domain.diameter / 2;

    % Initialize fluid region marker (1 = fluid everywhere initially)
    fluidMask = ones(ny+2, nx+2);

    % Generate obstacle perimeter points
    angularPos = linspace(0, 2*pi, 10000);
    cylX = obstacleX + obstacleRadius * cos(angularPos);
    cylY = obstacleY + obstacleRadius * sin(angularPos);

    % Mark solid region (points inside obstacle)
    if simCase ~= 1
        for j = 1:ny+2
            for i = 1:nx+2
                distanceToCenter = (gridX(i) - obstacleX)^2 + (gridY(j)
                    - obstacleY)^2;
                if distanceToCenter <= obstacleRadius^2
                    fluidMask(j, i) = 0;  % Mark as solid
                end
            end
        end
    end

end
```

A.2.4 Field Initialization

Listing 5: Initial stream function and density field setup

```
function [psi, density] = initializeFields(state, solver, fluidMask,
    varargin)
% INITIALIZEFIELDS Sets initial conditions for stream function and
    density
%
% Initializes computational fields with reference values and applies
% boundary conditions for solid/fluid regions.
%
% Inputs:
%   state     - Structure with initial values (psi_ref, density)
%   solver    - Structure containing nx and ny
%   fluidMask - Binary matrix indicating fluid (1) or solid (0) regions
%
% Outputs:
%   streamFunc - Initial stream function field (ny+2, nx+2)
%   density    - Initial density field (ny+2, nx+2)
%
    ------------------------------------------------------------------------


    nx = solver.nx;
    ny = solver.ny;

    % Initialize stream function with reference value everywhere
    psi = state.psi_ref * ones(ny+2, nx+2);

    % Initialize density: reference density in fluid, zero in solid
    density = state.density * fluidMask;

end
```

A.3 Discretization and Solver

A.3.1 Matrix Construction

Listing 6: Finite difference coefficient assembly and boundary conditions

```
function [a_E, a_W, a_N, a_S, a_P, rhs] = ...
         constructMatrix(psi, psi_boundary, solver, domain, velocity,
             gridX, gridY, fluidMask, state)
% CONSTRUCTMATRIX Builds coefficient matrices for discretized stream
    equation
%
% Constructs the finite difference stencil coefficients for solving the
```

```
% Laplace equation governing potential flow using Gauss-Seidel iteration.
%
% INPUTS:
%   streamFunc  - Current stream function field
%   psi_boundary - Stream function value on obstacle surface
%   solver      - Structure with nx, ny
%   domain      - Structure with length and height
%   velocity    - Inlet velocity
%   gridX, gridY - Coordinate vectors
%   fluidMask   - Binary matrix for solid/fluid regions
%   state       - Initial state structure
%
% OUTPUTS:
%   coeff_east, coeff_west, coeff_north, coeff_south - Face coefficients
%   coeff_center - Central point coefficient
%   rhs - Right-hand side vector
%
    ------------------------------------------------------------------------


    nx = solver.nx;
    ny = solver.ny;
    domainLength = domain.length;
    domainHeight = domain.height;

    dx = domainLength / nx;
    dy = domainHeight / ny;

    % Allocate coefficient matrices
    a_E = zeros(ny+2, nx+2);
    a_W = zeros(ny+2, nx+2);
    a_N = zeros(ny+2, nx+2);
    a_S = zeros(ny+2, nx+2);
    a_P = zeros(ny+2, nx+2);
    rhs = zeros(ny+2, nx+2);

    psi_bottom = state.psi_ref;

    % Interior nodes - construct finite difference stencil
    for j = 2:ny+1
        for i = 2:nx+1
            a_E(j, i) = dy / (gridX(i+1) - gridX(i));
            a_W(j, i) = dy / (gridX(i) - gridX(i-1));
            a_N(j, i) = dx / (gridY(j+1) - gridY(j));
            a_S(j, i) = dx / (gridY(j) - gridY(j-1));
            a_P(j, i) = a_E(j, i) + a_W(j, i) + ...
```

```
                                    a_N(j, i) + a_S(j, i);
            rhs(j, i) = 0;
        end
    end

    % Bottom boundary (j=1) - Dirichlet condition
    for i = 1:nx+2
        a_E(1, i) = 0;
        a_W(1, i) = 0;
        a_N(1, i) = 0;
        a_S(1, i) = 0;
        a_P(1, i) = 1;
        rhs(1, i) = psi_bottom;
    end

    % Top boundary (j=ny+2) - Dirichlet condition
    for i = 1:nx+2
        a_E(ny+2, i) = 0;
        a_W(ny+2, i) = 0;
        a_N(ny+2, i) = 0;
        a_S(ny+2, i) = 0;
        a_P(ny+2, i) = 1;
        rhs(ny+2, i) = psi_bottom + velocity * domainHeight;
    end

    % Inlet boundary (i=1) - Dirichlet condition
    for j = 2:ny+1
        a_E(j, 1) = 0;
        a_W(j, 1) = 0;
        a_N(j, 1) = 0;
        a_S(j, 1) = 0;
        a_P(j, 1) = 1;
        rhs(j, 1) = psi_bottom + velocity * gridY(j);
    end

    % Outlet boundary (i=nx+2) - Neumann-like condition
    for j = 2:ny+1
        a_E(j, nx+2) = 0;
        a_W(j, nx+2) = 1;
        a_N(j, nx+2) = 0;
        a_S(j, nx+2) = 0;
        a_P(j, nx+2) = 1;
        rhs(j, nx+2) = 0;
    end

    % Obstacle surface - Dirichlet condition
    for j = 2:ny+1
```

```
97          for i = 2:nx+1
98              if fluidMask(j, i) == 0
99                  a_E(j, i) = 0;
100                 a_W(j, i) = 0;
101                 a_N(j, i) = 0;
102                 a_S(j, i) = 0;
103                 a_P(j, i) = 1;
104                 rhs(j, i) = psi_boundary;
105             end
106         end
107     end
108
109 end
```

A.3.2 Iterative Solver

Listing 7: Gauss-Seidel method implementation

```
1  function [streamFunc, numIter, residual] = iterativeSolver(streamFunc,
       psi_boundary, fluidMask, ...
2                                                              a_E, a_W, a_N
                                                                   , ...
3                                                              a_S, a_P, rhs
                                                                   , ...
4                                                              velocity,
                                                                   gridY,
                                                                   solver,
                                                                   domain)
5  % ITERATIVESOLVER Solves Laplace equation using Gauss-Seidel iteration
6  %
7  % Implements the Gauss-Seidel iterative method with Dirichlet boundary
8  % conditions and internal solid region constraints.
9  %
10 % Inputs:
11 %   streamFunc  - Initial stream function field (guess)
12 %   psi_boundary - Stream function on obstacle surface
13 %   fluidMask   - Binary matrix for solid/fluid regions
14 %   coeff_*     - Finite difference coefficients
15 %   rhs         - Right-hand side vector
16 %   velocity    - Inlet velocity
17 %   gridY       - Y-coordinate vector
18 %   solver      - Structure with tolerance, max_iterations, nx, ny
19 %   domain      - Structure with height
20 %
21 % Outputs:
22 %   streamFunc - Converged stream function field
23 %   numIter    - Number of iterations performed
```

```
%   residual    - Final convergence error
%
    ---------------------------------------------------------------------------


    nx = solver.nx;
    ny = solver.ny;
    domainHeight = domain.height;
    tolerance = solver.tolerance;
    maxIterations = solver.max_iterations;

    residual = Inf;

    for numIter = 1:maxIterations
        streamFuncOld = streamFunc;

        % Apply inlet boundary condition
        for j = 1:ny+2
            streamFunc(j, 1) = velocity * gridY(j);
        end

        % Apply outlet boundary condition
        for j = 1:ny+2
            streamFunc(j, nx+2) = streamFunc(j, nx+1);
        end

        % Apply bottom wall boundary condition
        for i = 1:nx+2
            streamFunc(1, i) = 0;
        end

        % Apply top wall boundary condition
        for i = 1:nx+2
            streamFunc(ny+2, i) = velocity * domainHeight;
        end

        % Apply obstacle boundary condition
        for j = 1:ny+2
            for i = 1:nx+2
                if fluidMask(j, i) == 0
                    streamFunc(j, i) = psi_boundary;
                end
            end
        end

        % Gauss-Seidel iteration over interior nodes
        for j = 2:ny+1
```

```
            for i = 2:nx+1
                streamFunc(j, i) = (a_E(j, i) * streamFunc(j, i+1) + ...
                                    a_W(j, i) * streamFunc(j, i-1) + ...
                                    a_N(j, i) * streamFunc(j+1, i) + ...
                                    a_S(j, i) * streamFunc(j-1, i) + ...
                                    rhs(j, i)) / a_P(j, i);
            end
        end

        % Check convergence criterion
        residual = max(max(abs(streamFunc - streamFuncOld)));
        if residual < tolerance
            break;
        end
    end

end
```

A.4 Post-Processing and Analysis

A.4.1 Velocity Field Recovery

Listing 8: Velocity component reconstruction from stream function

```
function [velX, velY, velMag] = velocityField(streamFunc, gridX, gridY,
    solver, velocity, rho_ref, rho)
% VELOCITYFIELD Reconstructs velocity components from stream function
%
% Computes velocity field using the stream function through finite
% difference derivatives, applying the blocking-off method for density
    variation.
%
% INPUTS:
%   streamFunc  - Stream function values at grid points (ny+2, nx+2)
%   gridX, gridY - Coordinate vectors
%   solver      - Structure containing nx and ny
%   velocity    - Inlet velocity
%   rho_ref     - Reference density
%   rho         - Density field (ny+2, nx+2)
%
% OUTPUTS:
%   velX   - X-component of velocity (ny+2, nx+2)
%   velY   - Y-component of velocity (ny+2, nx+2)
%   velMag - Magnitude of velocity (ny+2, nx+2)
%
    ---------------------------------------------------------------------
```

```
20
21      nx = solver.nx;
22      ny = solver.ny;
23
24      % Initialize velocity arrays
25      velX = zeros(ny+2, nx+2);
26      velY = zeros(ny+2, nx+2);
27
28      % Precompute density ratios for blocking-off method
29      rho_ratio_east = zeros(ny+2, nx+2);
30      rho_ratio_west = zeros(ny+2, nx+2);
31      rho_ratio_north = zeros(ny+2, nx+2);
32      rho_ratio_south = zeros(ny+2, nx+2);
33
34      % Boundary conditions: No normal velocity at walls
35      for i = 1:nx+2
36          velY(1, i) = 0;       % Bottom wall
37          velY(ny+2, i) = 0;    % Top wall
38      end
39
40      % Outlet condition: Developed flow (zero gradient)
41      for j = 1:ny+2
42          velX(j, nx+2) = velX(j, nx+1);
43          velY(j, nx+2) = velY(j, nx+1);
44      end
45
46      % Inlet condition: Uniform velocity
47      for j = 1:ny+2
48          velX(j, 1) = velocity;
49          velY(j, 1) = 0;
50      end
51
52      % Compute velocity at interior nodes
53      for i = 2:nx+1
54          for j = 2:ny+1
55
56              % Compute density ratios (blocking-off method)
57              dx_east = gridX(i+1) - gridX(i);
58              dx_west = gridX(i) - gridX(i-1);
59              dy_north = gridY(j+1) - gridY(j);
60              dy_south = gridY(j) - gridY(j-1);
61
62              rho_ratio_east(j,i) = dx_east / ...
63                  (((0.5 * dx_east) / (rho_ref / rho(j,i))) + ...
64                   ((0.5 * dx_east) / (rho_ref / rho(j,i+1))));
65
```

```
            rho_ratio_west(j,i) = dx_west / ...
                (((0.5 * dx_west) / (rho_ref / rho(j,i))) + ...
                 ((0.5 * dx_west) / (rho_ref / rho(j,i-1))));

            rho_ratio_north(j,i) = dy_north / ...
                (((0.5 * dy_north) / (rho_ref / rho(j,i))) + ...
                 ((0.5 * dy_north) / (rho_ref / rho(j+1,i))));

            rho_ratio_south(j,i) = dy_south / ...
                (((0.5 * dy_south) / (rho_ref / rho(j,i))) + ...
                 ((0.5 * dy_south) / (rho_ref / rho(j-1,i))));

            % Compute velocity at cell faces
            % v_x = dpsi/dy, v_y = -dpsi/dx
            vel_north = +(1/rho_ratio_north(j,i)) * (streamFunc(j+1, i)
               - streamFunc(j, i)) / dy_north;
            vel_south = +(1/rho_ratio_south(j,i)) * (streamFunc(j, i) -
               streamFunc(j-1, i)) / dy_south;
            vel_east = -(1/rho_ratio_east(j,i)) * (streamFunc(j, i+1) -
               streamFunc(j, i)) / dx_east;
            vel_west = -(1/rho_ratio_west(j,i)) * (streamFunc(j, i) -
               streamFunc(j, i-1)) / dx_west;

            % Average face velocities to get node-centered values
            velX(j, i) =  (vel_north + vel_south) / 2;
            velY(j, i) = (vel_east + vel_west) / 2;

        end
    end

    % Compute velocity magnitude
    velMag = sqrt(velX.^2 + velY.^2);

end
```

A.4.2 Thermodynamic Properties

Listing 9: Temperature, pressure, and density calculation

```
function [tempField, pressField, rhoField, cpAvg] = thermoProperties(
   velMag, solver, state, material)
% THERMOPROPERTIES Calculates thermodynamic properties from flow field
%
% Computes temperature, pressure, and density using isentropic flow
   relations
% and polynomial model for specific heat capacity.
%
```

```
% INPUTS:
%   velMag  - Velocity magnitude field at grid points (ny+2, nx+2)
%   solver  - Structure containing nx and ny
%   state   - Structure with reference T, P, rho, V
%   material - Structure with R (gas constant) and gamma (heat ratio)
%
% OUTPUTS:
%   tempField - Temperature field (ny+2, nx+2)
%   pressField - Pressure field (ny+2, nx+2)
%   rhoField   - Density field (ny+2, nx+2)
%   cpAvg      - Average specific heat capacity field (ny+2, nx+2)
%
    ------------------------------------------------------------------------



    nx = solver.nx;
    ny = solver.ny;

    % Reference state values
    tempRef = state.temperature;
    pressRef = state.pressure;
    velRef = state.velocity;
    R_gas = material.R_gas;
    gamma = material.gamma;

    % Polynomial coefficients for cp(T) = A + B*T + C*T^2
    cp_A = 1005;         % J/(kg K) - intercept
    cp_B = 0.1;          % J/(kg K^2) - linear coefficient
    cp_C = -0.0002;      % J/(kg K^3) - quadratic coefficient

    % Initialize arrays
    tempField = zeros(ny+2, nx+2);
    pressField = zeros(ny+2, nx+2);
    rhoField = zeros(ny+2, nx+2);
    cpAvg = zeros(ny+2, nx+2);

    % Compute thermodynamic properties at interior nodes
    for i = 2:nx+1
        for j = 2:ny+1
            % Temperature using energy equation
            % T = T_ref + (V_ref^2 - V^2) / (2 * cp_mean)
            % For low-speed flow, use constant cp approximation
            cp_local = cp_A + cp_B * tempRef + cp_C * tempRef^2;

            % Temperature from energy equation (isentropic for ideal gas
               )
```

```
50              tempField(j,i) = tempRef + ((velRef^2 - velMag(j,i)^2) / (2
                    * cp_local));
51
52              % Guard against division by zero at stagnation points
53              deltaT = tempField(j,i) - tempRef;
54              if abs(deltaT) < 1e-10
55                  % At stagnation point or near-stagnation, use cp_A
                        directly
56                  cpAvg(j,i) = cp_A;
57              else
58                  % Compute average cp using integral definition
59                  % cpAvg = (1/ T ) *    [Tref to T] cp(  ) d
60                  cpAvg(j,i) = (1 / deltaT) * ...
61                               (cp_A * deltaT + ...
62                                (cp_B / 2) * (tempField(j,i)^2 - tempRef^2)
                                     + ...
63                                (cp_C / 3) * (tempField(j,i)^3 - tempRef^3)
                                    );
64              end
65
66              % For low-speed flow, temperature changes are typically
                    negligible
67              % No need for iterative refinement; single pass is
                    sufficient
68              % tempField(j,i) already computed above
69
70              % Pressure from isentropic relation: P = P_ref * (T/T_ref)^(
                    gamma/(gamma-1))
71              pressField(j,i) = pressRef * (tempField(j,i) / tempRef)^(
                    gamma / (gamma - 1));
72
73              % Density from ideal gas law: rho = P / (R * T)
74              rhoField(j,i) = pressField(j,i) / (R_gas * tempField(j,i));
75          end
76      end
77
78      % Apply boundary conditions to scalar fields (copy interior to
            boundaries)
79      % This ensures physical consistency at domain edges
80      tempField(1,:) = tempRef;
81      tempField(ny+2,:) = tempRef;
82      tempField(:,1) = tempRef;
83      tempField(:,nx+2) = tempRef;
84
85      pressField(1,:) = pressRef;
86      pressField(ny+2,:) = pressRef;
87      pressField(:,1) = pressRef;
```

```
88      pressField(:,nx+2) = pressRef;
89
90      rhoField(1,:) = state.density;
91      rhoField(ny+2,:) = state.density;
92      rhoField(:,1) = state.density;
93      rhoField(:,nx+2) = state.density;
94
95  end
```

A.4.3 Aerodynamic Coefficients

Listing 10: Force and circulation computation

```
1   function [coeffDrag, coeffLift, circulation, forceDrag, forceLift,
        pressureOnBody] = ...
2           aeroCoefficients(gridX, gridY, pressField, velX, velY, cylX,
                cylY, angularPos, domain, velocity)
3   % AEROCOEFFICIENTS Computes aerodynamic performance metrics
4   %
5   % Calculates drag and lift coefficients, forces, and circulation
6   % around a cylindrical obstacle using pressure and velocity fields.
7   % Uses Kutta-Joukowski theorem: L = rho * V_inf * Gamma
8   %
9   % INPUTS:
10  %   gridX, gridY      - Coordinate vectors
11  %   pressField        - Pressure field at grid points
12  %   velX, velY        - Velocity components at grid points
13  %   cylX, cylY        - Cylinder surface coordinate arrays
14  %   angularPos        - Angular positions around cylinder
15  %   domain            - Geometry structure (center, diameter, rho_ref)
16  %   rotSpeed          - Angular velocity (rad/s)
17  %   velocity          - Free-stream velocity
18  %
19  % OUTPUTS:
20  %   coeffDrag    - Drag coefficient
21  %   coeffLift    - Lift coefficient
22  %   circulation - Circulation around cylinder (Gamma)
23  %   forceDrag    - Drag force per unit depth
24  %   forceLift    - Lift force per unit depth
25  %   pressureOnBody - Pressure distribution on surface
26  %
    ----------------------------------------------------------------------

27
28      % Extract obstacle parameters
29      obstacleRadius = domain.diameter / 2;
30      rho_reference = domain.ref_density;
```

```
31
32      % Interpolate pressure on cylinder surface
33      pressureOnBody = interp2(gridX, gridY, pressField, cylX, cylY, '
            spline');
34
35      % Interpolate velocity components on cylinder surface
36      velX_surface = interp2(gridX, gridY, velX, cylX, cylY, 'spline');
37      velY_surface = interp2(gridX, gridY, velY, cylX, cylY, 'spline');
38
39      % Compute tangential velocity at surface (counter-clockwise positive
            )
40      % Normal vector points outward from center: (cos(theta), sin(theta))
41      % Tangent vector (counter-clockwise): (-sin(theta), cos(theta))
42      velTangential = -velX_surface .* sin(angularPos) + velY_surface .*
            cos(angularPos);
43
44      % Compute circulation from velocity line integral
45      % Gamma =      V dl around cylinder
46      circulation = trapz(angularPos, velTangential) * obstacleRadius;
47
48      % Compute aerodynamic forces using pressure integration
49      % Normal vector: n = (cos(theta), sin(theta))
50      % Pressure force: F = -    p * n dS
51      % Components: Fx = -    p*cos(theta) dS, Fy = -    p*sin(theta) dS
52
53      forceDrag = -trapz(angularPos, pressureOnBody .* cos(angularPos)) *
            obstacleRadius;
54      forceLift = -trapz(angularPos, pressureOnBody .* sin(angularPos)) *
            obstacleRadius;
55
56      % Reference dynamic pressure for non-dimensionalization
57      qRef = 0.5 * rho_reference * velocity^2;
58      referenceLength = domain.diameter;  % Diameter is characteristic
            length
59
60      % Compute dimensionless coefficients
61      coeffDrag = forceDrag / (qRef * referenceLength);
62      coeffLift = forceLift / (qRef * referenceLength);
63
64  end
```

A.5 Visualization

A.5.1 Flow Pattern Display

Listing 11: Streamline and flow pattern visualization

```
function displayFlowPattern(streamFunc, gridX, gridY)
% DISPLAYFLOWPATTERN Plots streamlines using stream function contours
%
% Generates a streamline visualization by contouring the stream function
% values over a computational grid.
%
% Inputs:
%   streamFunc - Stream function values at grid points (ny+2, nx+2)
%   gridX      - X-coordinate vector
%   gridY      - Y-coordinate vector
%
% Outputs:
%   Graphical streamline contour plot
%
% Notes:
%   Creates 30 streamline contours in black color
%
    ------------------------------------------------------------------------


    [meshX, meshY] = meshgrid(gridX, gridY);
    contour(meshX, meshY, streamFunc, 30, 'k', 'LineWidth', 0.01);

end
```

A.5.2 Scalar Field Visualization

Listing 12: Pressure, velocity, and temperature field rendering

```
function visualizeScalarField(fieldLabel, fieldData, nx, ny, dx, dy)
% VISUALIZESCALARFIELD Creates a color-mapped visualization of a scalar
    field
%
% Generates a graphical representation of a 2D scalar field using
% interpolated shading and color mapping.
%
% Inputs:
%   fieldLabel - Title/label for the field (string)
%   fieldData  - 2D matrix of scalar values (ny+2, nx+2)
%   nx         - Number of grid points in x-direction
%   ny         - Number of grid points in y-direction
%   dx         - Grid spacing in x-direction
%   dy         - Grid spacing in y-direction
%
% No explicit outputs (creates figure)
```

```
%
    ------------------------------------------------------------------------


    % Display field with interpolated shading
    imagesc((0.5:nx-0.5) * dx, (0.5:ny-0.5) * dy, fieldData(2:end-1, 2:
        end-1));
    axis xy equal tight;
    cb = colorbar();
    cb.Label.String = fieldLabel;
    shading interp;
    colormap jet;

    % Configure axes and labels
    xlabel('x [m]');
    ylabel('y [m]');
    grid off;

    % Set title
    if isempty(fieldLabel)
        title('Scalar Field', 'FontWeight', 'bold', 'FontSize', 12);
    else
        title(['Field: ', fieldLabel], 'FontWeight', 'bold', 'FontSize',
            12);
    end

end
```

A.6 Validation and Utilities

A.6.1 Convergence Validation

Listing 13: Convergence study and error analysis

```
%% Numerical Convergence Analysis and Validation Study
% Validates numerical solutions against analytical solutions with mesh
    refinement
% Tests convergence rates and tolerance effects
%
% Tests convergence for case 1 (channel) across different mesh
    resolutions
%
clearvars; close all; clc;

%% Configuration Phase
```

```
gridSizes = [8, 16, 32, 64, 128, 256];  % Mesh refinement levels: 2^3 to 2^8
numCases = 1;
numGrids = length(gridSizes);

% Data storage structure
convergenceMetrics = struct();
convergenceMetrics.grid_sizes = gridSizes;
convergenceMetrics.error_L2 = zeros(numCases, numGrids);
convergenceMetrics.error_Linf = zeros(numCases, numGrids);
convergenceMetrics.spacing_dx = zeros(1, numGrids);
convergenceMetrics.psi_analytical_ref = cell(numCases, numGrids);
convergenceMetrics.psi_numerical_ref = cell(numCases, numGrids);
convergenceMetrics.solver_iterations = zeros(numCases, numGrids);
convergenceMetrics.solver_residual = zeros(numCases, numGrids);

fprintf('========= CONVERGENCE ANALYSIS FRAMEWORK =========\n');
fprintf('Grid sizes (N, M): %s\n', sprintf('%d ', gridSizes));
fprintf('Cases: 1=Channel, 2=Static Cylinder, 3=Rotating Cylinder\n\n');

%% Main Convergence Refinement Loop
for caseIdx = 1:numCases
    fprintf('---------- CASE %d CONVERGENCE ----------\n', caseIdx);

    for gridIdx = 1:numGrids
        N = gridSizes(gridIdx);
        M = gridSizes(gridIdx);

        fprintf('  Mesh: %d  %d ... ', N, M);
        timeStart = tic;

        % Initialize Simulation Parameters
        [domainGeo, flowInlet, flowBoundary, fluidMaterial, numericalCfg, ...
         initialState, cylinderSpeed, flowCase] = setupSimulation();

        numericalCfg.nx = N;
        numericalCfg.ny = M;
        flowCase = caseIdx;

        % Recompute boundary condition for current case
        if flowCase == 1
            psiBoundary = 0;  % Open channel
        else
            r_cyl = domainGeo.diameter / 2;
            if flowCase == 2
                psiBoundary = flowInlet.velocity * (domainGeo.height/2);
```

```
55              else  % flowCase == 3
56                  psiBoundary = flowInlet.velocity * (domainGeo.height/2)
                        + ...
57                              r_cyl^2 * cylinderSpeed * (1/(2*pi));
58              end
59          end
60
61          % Mesh Generation
62          [meshX, meshY] = createComputationalGrid(domainGeo, numericalCfg
                );
63
64          % Obstacle Definition
65          if flowCase == 1
66              % Channel without obstacle
67              regionFluid = ones(numericalCfg.ny+2, numericalCfg.nx+2);
68              obstacleX = [];
69              obstacleY = [];
70              anglePos = [];
71          else
72              [obstacleX, obstacleY, regionFluid, anglePos] = ...
73                  defineObstacles(domainGeo, meshX, meshY, numericalCfg);
74          end
75
76          % Field Initialization
77          [psiField, rhoField] = initializeFields(initialState,
                numericalCfg, regionFluid);
78
79          % Discretization Matrix Construction
80          [coeff_e, coeff_w, coeff_n, coeff_s, coeff_c, rhs_vec] = ...
81              constructMatrix(psiField, psiBoundary, numericalCfg,
                    domainGeo, ...
82                              flowInlet.velocity, meshX, meshY, regionFluid
                                    , initialState);
83
84          % Solve Stream Function
85          [psiField, numIterations, finalResidual] = iterativeSolver(
                psiField, psiBoundary, ...
86                                                                  regionFluid
                                                                        ,
                                                                    coeff_e
                                                                        ,
                                                                    coeff_w
                                                                        ,
                                                                    ...
87                                                                  coeff_n
                                                                        ,
```

```
                                                                    coeff_s
                                                                    ,
                                                                    coeff_c
                                                                    ,
                                                                    rhs_vec
                                                                    ,
                                                                    ...

88                                                                flowInlet
                                                                    .
                                                                    velocity
                                                                    ,
                                                                    meshY
                                                                    ,
                                                                    numericalCfg
                                                                    ,
                                                                    domainGeo
                                                                    );
89
90          % Reference Analytical Solution
91          radiusObs = domainGeo.diameter / 2;
92          psiAnalyticalRef = streamFunctionAnalytical(meshX, meshY,
                domainGeo, ...
93                                                      flowInlet.velocity,
                                                            flowCase, ...
94                                                      cylinderSpeed,
                                                            radiusObs);
95
96          % Error Metric Calculation
97          if flowCase ~= 1
98              validRegion = regionFluid == 1;  % Only compute error in
                    fluid domain
99          else
100             validRegion = ones(size(regionFluid));  % All domain for
                    channel
101         end
102
103         % L2-norm Error
104         errorDomain = (psiField - psiAnalyticalRef) .* validRegion;
105         errorL2 = sqrt(sum(sum(errorDomain.^2)) / sum(sum(validRegion)))
                ;
106
107         % L-infinity Norm Error
108         errorLinf = max(max(abs(errorDomain)));
109
110         % Store Results
111         convergenceMetrics.error_L2(caseIdx, gridIdx) = errorL2;
```

```
112         convergenceMetrics.error_Linf(caseIdx, gridIdx) = errorLinf;
113         convergenceMetrics.spacing_dx(gridIdx) = domainGeo.length / N;
114         convergenceMetrics.psi_numerical_ref{caseIdx, gridIdx} =
                psiField;
115         convergenceMetrics.psi_analytical_ref{caseIdx, gridIdx} =
                psiAnalyticalRef;
116         convergenceMetrics.solver_iterations(caseIdx, gridIdx) =
                numIterations;
117         convergenceMetrics.solver_residual(caseIdx, gridIdx) =
                finalResidual;
118
119         timeElapsed = toc(timeStart);
120         fprintf('L2-error=%.2e,  L  -error=%.2e, Iter=%d, Time=%.2fs\n',
                ...
121             errorL2, errorLinf, numIterations, timeElapsed);
122     end
123     fprintf('\n');
124 end
125
126 %% Convergence Rate Computation
127 fprintf('=== CONVERGENCE RATE ANALYSIS ===\n');
128 fprintf('Case | L2 Order |  L   Order | Expected (2nd-order scheme)\n');
129 fprintf('-----|----------|----------|---------------------------\n');
130
131 for caseIdx = 1:numCases
132     if convergenceMetrics.error_L2(caseIdx, 1) > 0 && convergenceMetrics
            .error_L2(caseIdx, 2) > 0
133         dx1 = convergenceMetrics.spacing_dx(1);
134         dx2 = convergenceMetrics.spacing_dx(2);
135         e1 = convergenceMetrics.error_L2(caseIdx, 1);
136         e2 = convergenceMetrics.error_L2(caseIdx, 2);
137
138         if e2 > 0
139             orderL2 = log(e2/e1) / log(dx2/dx1);
140         else
141             orderL2 = NaN;
142         end
143
144         if convergenceMetrics.error_Linf(caseIdx, 1) > 0 &&
                convergenceMetrics.error_Linf(caseIdx, 2) > 0
145             e1_inf = convergenceMetrics.error_Linf(caseIdx, 1);
146             e2_inf = convergenceMetrics.error_Linf(caseIdx, 2);
147             orderLinf = log(e2_inf/e1_inf) / log(dx2/dx1);
148         else
149             orderLinf = NaN;
150         end
151
```

```
152         fprintf('%d    |  %.2f    |   %.2f   |           2.0\n', caseIdx
                , orderL2, orderLinf);
153     end
154 end
155
156 %% Visualization: Convergence Performance
157 figPos = [100, 100, 900, 700];
158
159 % Error vs Grid Spacing (L2-norm)
160 figure('Name', 'L2-norm Convergence (Case 1)', 'NumberTitle', 'off', '
    Position', figPos);
161 hold on;
162 dxRef = convergenceMetrics.spacing_dx;
163 loglog(convergenceMetrics.spacing_dx, convergenceMetrics.error_L2(1, :),
     ...
164         'o-', 'LineWidth', 2.5, 'MarkerSize', 8, 'DisplayName', '
            Numerical');
165
166 % Reference convergence lines
167 eRef1st = convergenceMetrics.error_L2(1, 1) * (dxRef / dxRef(1)).^1;
168 eRef2nd = convergenceMetrics.error_L2(1, 1) * (dxRef / dxRef(1)).^2;
169 loglog(dxRef, eRef1st, 'k--', 'LineWidth', 1.5, 'DisplayName', '1st
    Order');
170 loglog(dxRef, eRef2nd, 'k:', 'LineWidth', 1.5, 'DisplayName', '2nd Order
    ');
171 hold off;
172 xlabel('Grid Spacing (dx)');
173 ylabel('L2-norm Error');
174 title('L2-norm Convergence vs Mesh Refinement (Case 1 - Channel Flow)');
175 legend('FontSize', 11, 'Location', 'best');
176 grid on;
177 set(gca, 'XDir', 'reverse');
178
179 % Error vs Grid Spacing (L-infinity norm)
180 figure('Name', ' L  -norm Convergence (Case 1)', 'NumberTitle', 'off', '
    Position', figPos);
181 hold on;
182 loglog(convergenceMetrics.spacing_dx, convergenceMetrics.error_Linf(1,
    :), ...
183         's-', 'LineWidth', 2.5, 'MarkerSize', 8, 'DisplayName', '
            Numerical');
184
185 eRef1stInf = convergenceMetrics.error_Linf(1, 1) * (dxRef / dxRef(1))
    .^1;
186 eRef2ndInf = convergenceMetrics.error_Linf(1, 1) * (dxRef / dxRef(1))
    .^2;
```

```
187 loglog(dxRef, eRef1stInf, 'k--', 'LineWidth', 1.5, 'DisplayName', '1st
    Order');
188 loglog(dxRef, eRef2ndInf, 'k:', 'LineWidth', 1.5, 'DisplayName', '2nd
    Order');
189 hold off;
190 xlabel('Grid Spacing (dx)');
191 ylabel(' L  -norm Error');
192 title(' L  -norm Convergence vs Mesh Refinement (Case 1 - Channel Flow)'
    );
193 legend('FontSize', 11, 'Location', 'best');
194 grid on;
195 set(gca, 'XDir', 'reverse');
196
197 % Solver Iterations vs Mesh Size
198 figure('Name', 'Iterations vs Mesh Refinement (Case 1)', 'NumberTitle',
    'off', 'Position', figPos);
199 semilogy(gridSizes, convergenceMetrics.solver_iterations(1, :), ...
200          'o-', 'LineWidth', 2.5, 'MarkerSize', 8);
201 xlabel('Mesh Size (N = M)');
202 ylabel('Number of Iterations');
203 title('Solver Convergence Iterations vs Mesh Refinement (Case 1)');
204 legend('Case 1');
205 grid on;
206 set(gca, 'XScale', 'log');
207
208 % Solver Residual vs Mesh Size
209 figure('Name', 'Solver Residual vs Mesh Refinement (Case 1)', '
    NumberTitle', 'off', 'Position', figPos);
210 loglog(gridSizes, convergenceMetrics.solver_residual(1, :), ...
211        'd-', 'LineWidth', 2.5, 'MarkerSize', 8);
212 xlabel('Mesh Size (N = M)');
213 ylabel('Final Residual');
214 title('Solver Residual vs Mesh Refinement (Case 1)');
215 legend('Case 1');
216 grid on;
217 set(gca, 'XScale', 'log');
218
219 %% Summary Table
220 fprintf('\n=== CONVERGENCE SUMMARY TABLE ===\n');
221 fprintf('Grid  | Spacing  | L2-Error   |  L  -Error   | Iterations |
    Residual\n');
222 fprintf('
    ------|----------|------------|------------|------------|----------\n
    ');
223 for gridIdx = 1:numGrids
224     fprintf('%3d   | %.2e | %.2e | %.2e |    %d      | %.2e\n', ...
225         gridSizes(gridIdx), convergenceMetrics.spacing_dx(gridIdx), ...
```

```
        convergenceMetrics.error_L2(1, gridIdx), ...
        convergenceMetrics.error_Linf(1, gridIdx), ...
        convergenceMetrics.solver_iterations(1, gridIdx), ...
        convergenceMetrics.solver_residual(1, gridIdx));
end

%% Save Results
save('convergence_analysis_results.mat', 'convergenceMetrics');
fprintf('\n\nAnalysis results saved to: convergence_analysis_results.mat\n');
fprintf('Convergence validation study completed successfully!\n');
```

A.6.2 Analytical Stream Function

Listing 14: Analytical solutions for potential flow validation

```
function psiAnalytical = streamFunctionAnalytical(gridX, gridY, domain, velocity, simCase, rotSpeed, obstacleRadius)
% STREAMFUNCTIONANALYTICAL Computes analytical stream function solutions
%
% Calculates the analytical stream function for three different flow configurations:
% - Case 1: Uniform channel flow
% - Case 2: Static obstacle with potential flow
% - Case 3: Rotating obstacle with circulation
%
% INPUTS:
%   gridX, gridY        - 1D mesh coordinate arrays
%   domain              - Geometry structure with center coordinates and dimensions
%   velocity            - Inlet flow velocity
%   simCase             - Case selector (1, 2, or 3)
%   rotSpeed            - Angular velocity for rotating case
%   obstacleRadius      - Radius of cylindrical obstacle
%
% OUTPUTS:
%   psiAnalytical - Analytical stream function field (ny+2, nx+2)
%
    ------------------------------------------------------------------------



    ny = length(gridY) - 2;
    nx = length(gridX) - 2;
    psiAnalytical = zeros(ny+2, nx+2);

    centerX = domain.center_x;
    centerY = domain.center_y;
```

```
    domainHeight = domain.height;
    domainLength = domain.length;

    % Create mesh grid for computation
    [meshX, meshY] = meshgrid(gridX, gridY);

    switch simCase
        case 1
            % CASE 1: Uniform channel flow (simple linear profile)
            % psi(x, y) = velocity * y
            psiAnalytical = velocity * meshY;

        case 2
            % CASE 2: Potential flow around static obstacle (uniform + doublet)
            % psi = psi_uniform + psi_doublet

            % Distance from obstacle center
            radiusSq = (meshX - centerX).^2 + (meshY - centerY).^2;
            radius = sqrt(radiusSq);

            % Uniform flow component
            psiUniform = velocity * meshY;

            % Doublet strength representing the obstacle effect
            doubletStrength = 2 * pi * obstacleRadius^2 * velocity;

            % Doublet component (only outside obstacle)
            psiDoublet = zeros(size(meshX));
            outerRegion = radius > obstacleRadius;
            psiDoublet(outerRegion) = (doubletStrength / (2*pi)) * ...
                                      (meshY(outerRegion) - centerY) ./ ...
                                         radiusSq(outerRegion);

            % Total solution: uniform + doublet
            psiAnalytical = psiUniform + psiDoublet;

            % Inside obstacle: constant stream function
            psiAnalytical(radius <= obstacleRadius) = velocity * centerY;

        case 3
            % CASE 3: Rotating obstacle with circulation (static + vortex)
            % psi = psi_static + psi_circulation

            radius = sqrt((meshX - centerX).^2 + (meshY - centerY).^2);
```

```
            radiusSq = radius.^2;

            % Static component (uniform + doublet)
            doubletStrength = 2 * pi * obstacleRadius^2 * velocity;
            psiUniform = velocity * meshY;

            psiStatic = psiUniform;
            outerRegion = radius > obstacleRadius;
            psiStatic(outerRegion) = psiStatic(outerRegion) + ...
                                     (doubletStrength / (2*pi)) * ...
                                     (meshY(outerRegion) - centerY) ./
                                        radiusSq(outerRegion);
            psiStatic(radius <= obstacleRadius) = velocity * centerY;

            % Circulation component from rotation
            circulationStrength = 2 * pi * obstacleRadius^2 * rotSpeed;
            psiCirculation = zeros(size(meshX));
            psiCirculation(outerRegion) = -(circulationStrength / (2*pi)
                ) * ...
                                          log(radius(outerRegion) /
                                             obstacleRadius);

            % Total solution: static + circulation effect
            psiAnalytical = psiStatic + psiCirculation;
        end

end
```